

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

PYTHON_TEST

- load_display_save.py
- masked_histogram_co...
- masked_result.png
- masked.png
- my_masterpiece.png
- noise_reduction_pipeli...
- output.bmp
- output.jpg
- output.tiff
- panorama_prep.png
- pixel_basics.py
- red.jpg
- region_comparison.png
- resize.py
- rotate.py
- saved_image.jpg
- saved_image.png
- simple_thresholding.py
- smart_crop.png
- target.png
- terex.png
- test_gradient.png
- test_image_gray.png
- test_image.png
- test_pattern.png
- thumbnail_gallery.png
- transformed.png
- translation.py
- value_gradient.png
- verify.py
- white.jpg

simple_thresholding.py > ...

```
1 # USAGE
2 # python simple_thresholding.py --image ../images/coins.png
3
4 # Import the necessary packages
5 import numpy as np
6 import argparse
7 import cv2
8
9 # Construct the argument parser and parse the arguments
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-i", "--image", required = True,
12                 help = "Path to the image")
13 args = vars(ap.parse_args())
14
15 # Load the image, convert it to grayscale, and blur it slightly
16 image = cv2.imread(args["image"])
17 image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
18 blurred = cv2.GaussianBlur(image, (5, 5), 0)
19 cv2.imshow("Image", image)
20
21 # Let's apply basic thresholding. The first parameter is the
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\raghu\OneDrive\Desktop\python_test> python simple_thresholding.py --image cat.png
PS C:\Users\raghu\OneDrive\Desktop\python_test>
```



File Edit Selection View Go Run Terminal Help python_test

EXPLORER

PYTHON_TEST

- load_display_save.py
- masked_histogram_co...
- masked_result.png
- masked.png
- my_masterpiece.png
- noise_reduction_pipeli...
- otsu_and_riddler.py
- output.bmp
- output.jpg
- output.tiff
- panorama_prep.png
- pixel_basics.py
- red.jpg
- region_comparison.png
- resize.py
- rotate.py
- saved_image.jpg
- saved_image.png
- simple_thresholding.py
- smart_crop.png
- target.png
- terex.png
- test_gradient.png
- test_image_gray.png
- test_image.png
- test_pattern.png
- thumbnail_gallery.png
- transformed.png
- translation.py
- value_gradient.png
- verify.py
- white.jpg

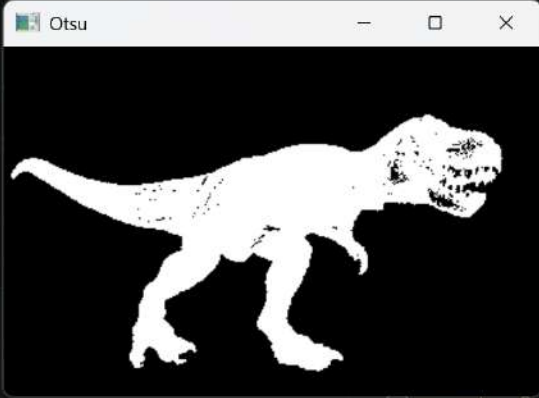
otsu_and_riddler.py > ...

```
1 # USAGE
2 # python otsu_and_riddler.py --image ../images/coins.png
3
4 # Import the necessary packages
5 from __future__ import print_function
6 import numpy as np
7 import argparse
8 import mahotas
9 import cv2
10
11 # Construct the argument parser and parse the arguments
12 ap = argparse.ArgumentParser()
13 ap.add_argument("-i", "--image", required = True,
14               help = "Path to the image")
15 args = vars(ap.parse_args())
16
17 # Load the image, convert it to grayscale, and blur it slightly
18 image = cv2.imread(args["image"])
19 image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
20 blurred = cv2.GaussianBlur(image, (5, 5), 0)
21 cv2.imshow("Image", image)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python otsu_and_riddler.py --image terex.png
Otsu's threshold: 182
Riddler-Calvard: 182.2638597924003
[]

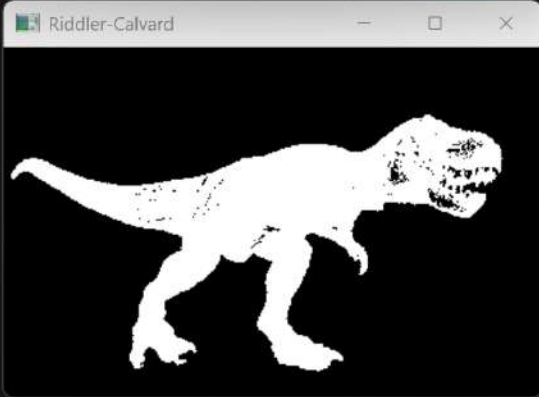
Otsu



Image



Riddler-Calvard



Ln 2, Col 29 (19 selected) Spaces: 4 UTF-8 CRLF {} Python 3.14.5 Python 3.14 (64-bit)

File Edit Selection View Go Run Terminal Help python_test

EXPLORER

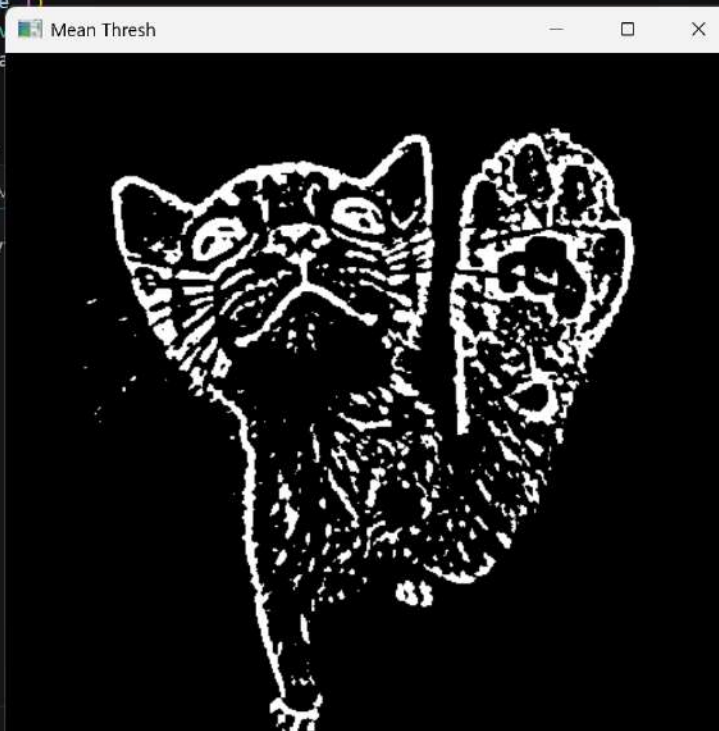
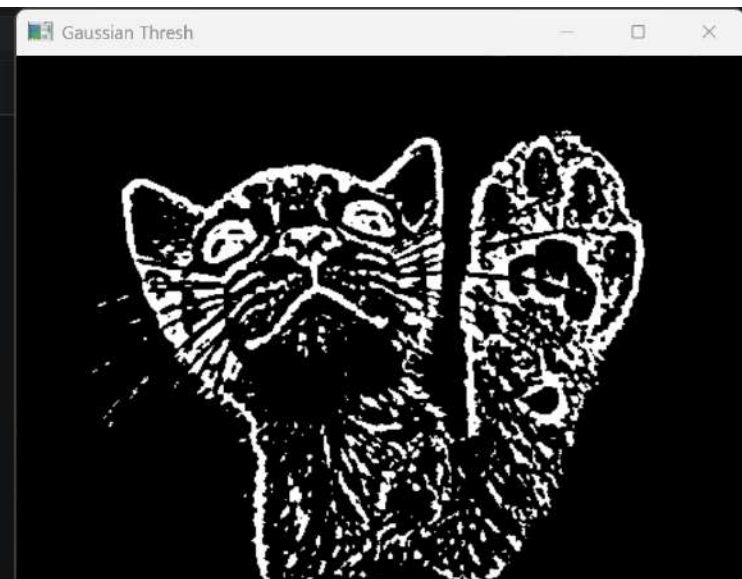
- PYTHON_TEST
 - __pycache__
 - .vscode
 - settings.json
 - day7_output
 - New folder
 - 2d_color_projections.p...
 - 2d_histograms.png
 - 3d_color_distribution....
 - 3d_histogram_projecti...
 - adaptive_thresholding...
 - annotated_image.jpg
 - annotated.png
 - art_cyanotype.png
 - art_green_screen.png
 - art_magenta_dream.p...
 - art_sepia.png
 - before_after.png
 - black.jpg
 - blended.png
 - blurring.py
 - cat.png
 - challenge_blend.png
 - challenge_main.png
 - clock.png
 - color_histograms.png
 - color_histograms.py
 - color_isolation_blue.png
 - color_isolation_green....
 - color_isolation_purple....
 - color_isolation_red.png
 - color_isolation_yellow....
 - color_splash.png
- OUTLINE
- TIMELINE

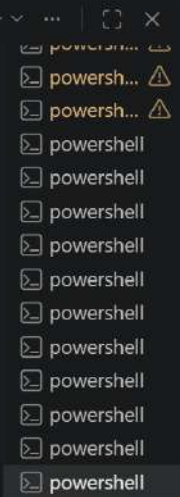
adaptive_thresholding.py

```
1 # USAGE
2 # python adaptive_thresholding.py --image ../images/coins.png
3
4 # Import the necessary packages
5 import numpy as np
6 import argparse
7 import cv2
8
9 # Construct the argument parser and parse the arguments
10 ap = argparse.ArgumentParser()
11 ap.add_argument("-i", "--image", required = True,
12                 help = "Path to the image")
13 args = vars(ap.parse_args())
14
15 # Load the image, convert it to grayscale, and blur it slightly
16 image = cv2.imread(args["image"])
17 image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
18 blurred = cv2.GaussianBlur(image, (5, 5), 0)
19 cv2.imshow("Image", image)
20
21 # In our previous example, we
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

PS C:\Users\raghu\OneDrive\Desktop\python_test>





File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - day8_color_spaces.py
 - day9_histograms.py
 - day10_smoothing_blu...
 - day11_thresholding.py
 - dissolve.mp4
 - drawing_basics.py
 - drawing.py
 - edge_preserving_com...
 - enhanced_image.png
 - equalization_comparis...
 - equalize.py
 - equalized_image.png
 - exercise_1_channel_art...
 - exercise_1_custom_pip...
 - exercise_1_histogram_...
 - exercise_1_interactive_...
 - exercise_1_threshold_...
 - exercise_2_color_picke...
 - exercise_2_compariso...
 - exercise_2_equalizatio...
 - exercise_2_noise_redu...
 - exercise_3_3d_color_hi...
 - exercise_3_color_isolat...
 - exercise_3_custom_ker...
 - exercise_4_color_spac...
 - exercise_4_edge_prese...
 - exercise_4_histogram_...
 - exercise_5_contrast_en...
 - exercise_5_deblurring....
 - exercise1.brighthness.c...
 - exercise1.day3.py
 - exercise1.house.py
 - exercise1.ny

exercise_1_threshold_optimizer.py

```
1 #exercise_1_threshold_optimizer.py
2 import cv2
3 import numpy as np
4
5 def threshold_optimizer(image_path):
6     """Find optimal threshold by analyzing histogram"""
7     image = cv2.imread(image_path)
8     if image is None:
9         print(f"ERROR: Could not load image from {image_path}!")
10        return None
11
12    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
13
14    # Compute histogram
15    hist = cv2.calcHist([gray], [0], None, [256], [0, 256])
16    hist = hist.flatten()
17
18    # Find distinct peaks in histogram (filtering out minor noise)
19    peaks = []
20    avg_hist_val = np.mean(hist)
21
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python exercise_1_threshold_optimizer.py
Detected two major peaks at 76 and 150
Optimal threshold T = 77
█

Threshold Optimizer

Original Optimal T = 77

EXPLORER

PYTHON_TEST

- day8_color_spaces.py
- day9_histograms.py
- day10_smoothing_blu...
- day11_thresholding.py
- dissolve.mp4
- drawing_basics.py
- drawing.py
- edge_preserving_com...
- enhanced_image.png
- equalization_comparis...
- equalize.py
- equalized_image.png
- exercise_1_channel_art...
- exercise_1_custom_pip...
- exercise_1_histogram_...
- exercise_1_interactive_...
- exercise_1_threshold_...
- exercise_2_color_picke...
- exercise_2_compariso...
- exercise_2_document...**
- exercise_2_equalizatio...
- exercise_2_noise_redu...
- exercise_3_3d_color_hi...
- exercise_3_color_isolat...
- exercise_3_custom_ker...
- exercise_4_color_spac...
- exercise_4_edge_prese...
- exercise_4_histogram_...
- exercise_5_contrast_en...
- exercise_5_deblurring...
- exercise1.brighthness.c...
- exercise1.day3.py
- exercise1.house.py

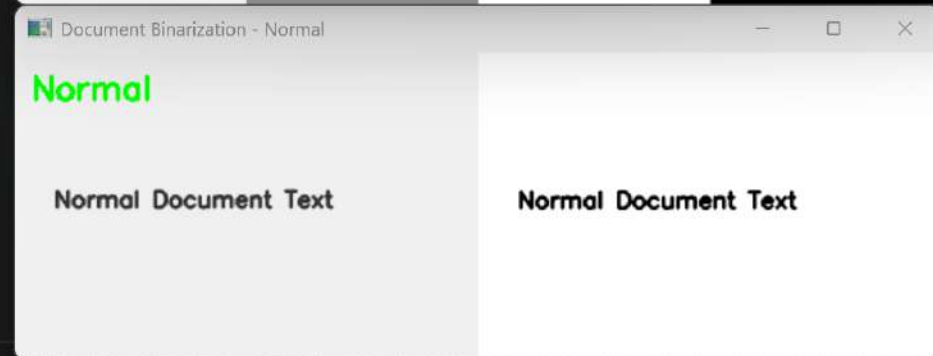
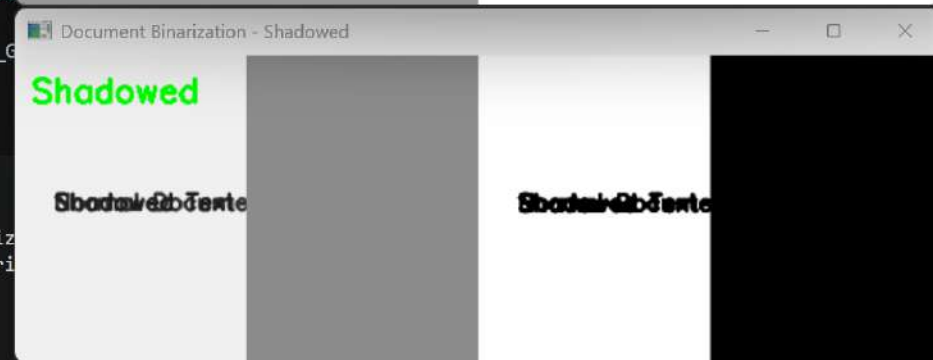
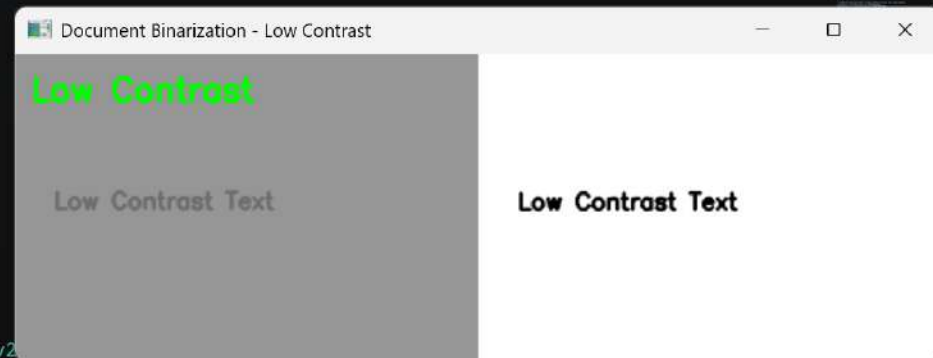
WELCOME

exercise_2_document_binarization.py

```
1 # exercise_2_document_binarization.py
2 import cv2
3 import numpy as np
4
5 def binarize_document(image, method="auto"):
6     """ Binarize a document image for OCR preprocessing """
7     if len(image.shape) == 3:
8         gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
9     else:
10         gray = image.copy()
11
12     # Remove noise with blur
13     blurred = cv2.GaussianBlur(gray, (3, 3), 0)
14
15     if method == "simple":
16         _, binary = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
17     elif method == "adaptive":
18         binary = cv2.adaptiveThreshold(blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C, cv2.THRESH_BINARY, 11, 2)
19     else:
20         # auto - choose based on image characteristics
21         hist = cv2.calcHist([blurred], [0], None, [256], [0, 256])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python exercise_2_document_binarization.py



File Edit Selection View Go Run Terminal Help python_test

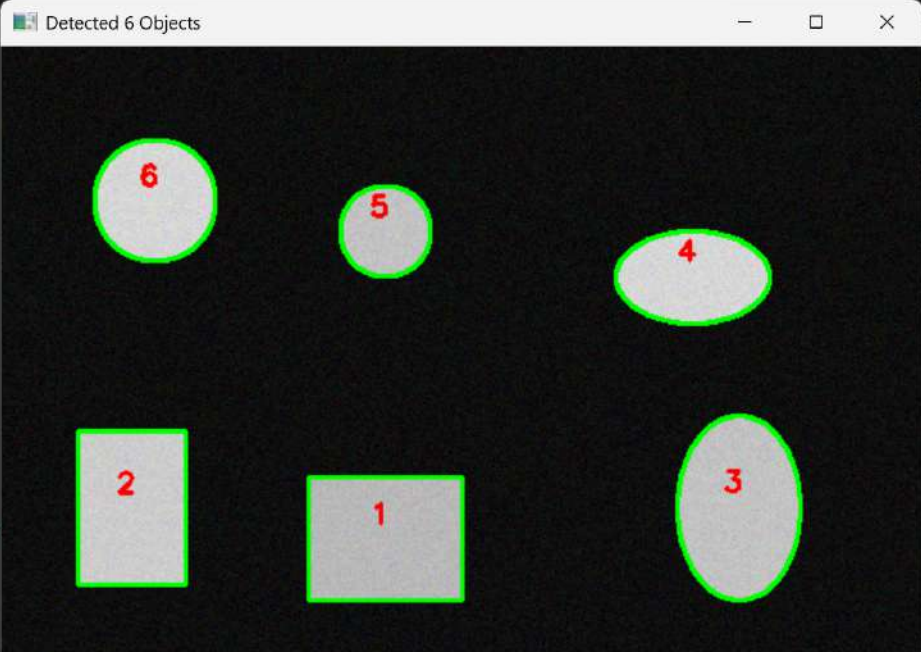
EXPLORER

- PYTHON_TEST
 - day8_color_spaces.py
 - day9_histograms.py
 - day10_smoothing_blu...
 - day11_thresholding.py
 - dissolve.mp4
 - drawing_basics.py
 - drawing.py
 - edge_preserving_com...
 - enhanced_image.png
 - equalization_comparis...
 - equalize.py
 - equalized_image.png
 - exercise_1_channel_art...
 - exercise_1_custom_pip...
 - exercise_1_histogram...
 - exercise_1_interactive...
 - exercise_1_threshold...
 - exercise_2_color_picke...
 - exercise_2_compariso...
 - exercise_2_document...
 - exercise_2_equalizatio...
 - exercise_2_noise_redu...
 - exercise_3_3d_color_hi...
 - exercise_3_color_isolat...
 - exercise_3_custom_ker...
 - exercise_3_multi_objec...**
 - exercise_4_color_spac...
 - exercise_4_edge_prese...
 - exercise_4_histogram...
 - exercise_5_contrast_en...
 - exercise_5_deblurring...
 - exercise1.brighthness.c...
 - exercise1.day3.mv
- OUTLINE
- TIMELINE

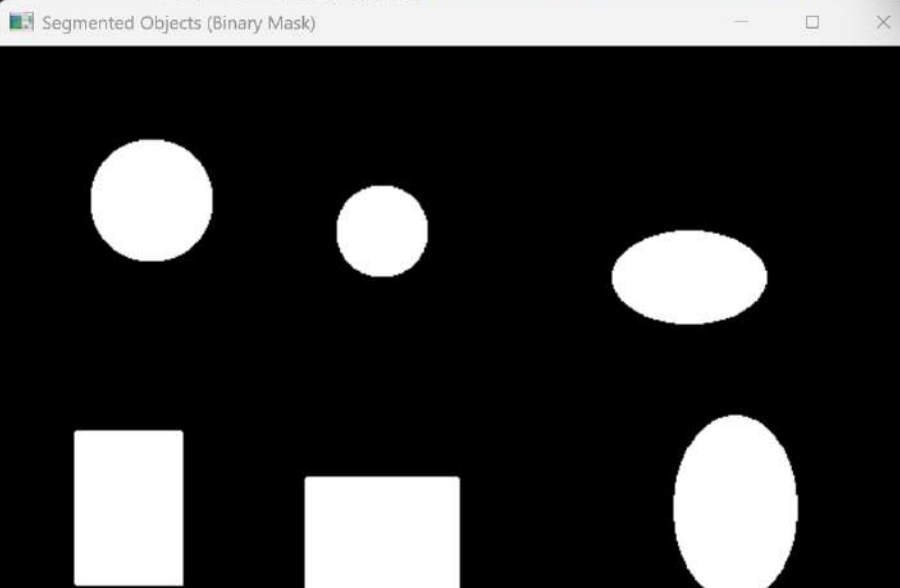
exercise_3_multi_object_segmentation.py > ...

```
1 # exercise_3_multi_object_segmentation.py
2 import cv2
3 import numpy as np
4
5 def segment_objects(image, min_area=100):
6     """ Segment objects from background using thresholding """
7     if len(image.shape) == 3:
8         gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
9     else:
10        gray = image.copy()
11
12    # Apply blur
13    blurred = cv2.GaussianBlur(gray, (5, 5), 0)
14
15    # Otsu threshold
16    _, binary = cv2.threshold(blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
17
18    # Clean up minor noise artifacts from background before tracking
19    kernel = np.ones((3, 3), np.uint8)
20    binary = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
```

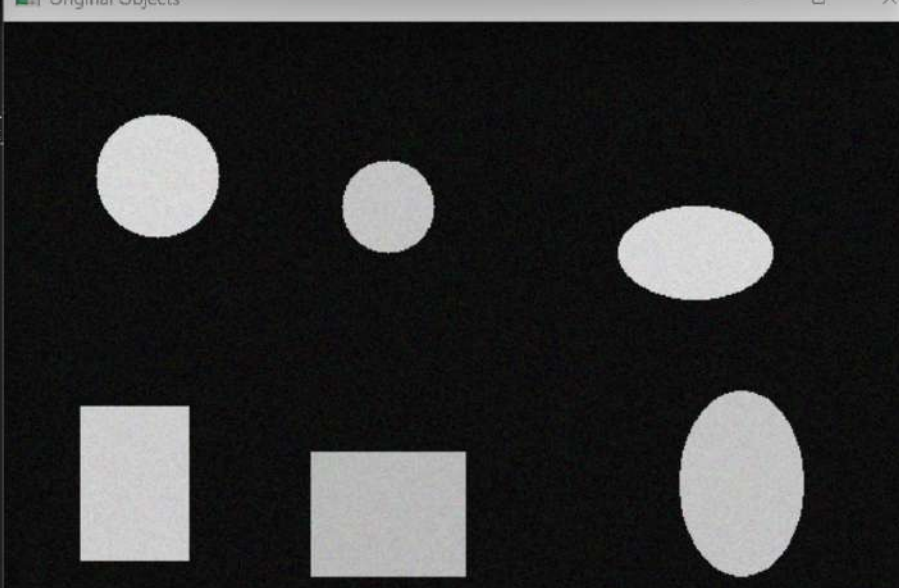
Detected 6 Objects



Segmented Objects (Binary Mask)



Original Objects



File Edit Selection View Go Run Terminal Help python_test

EXPLORER

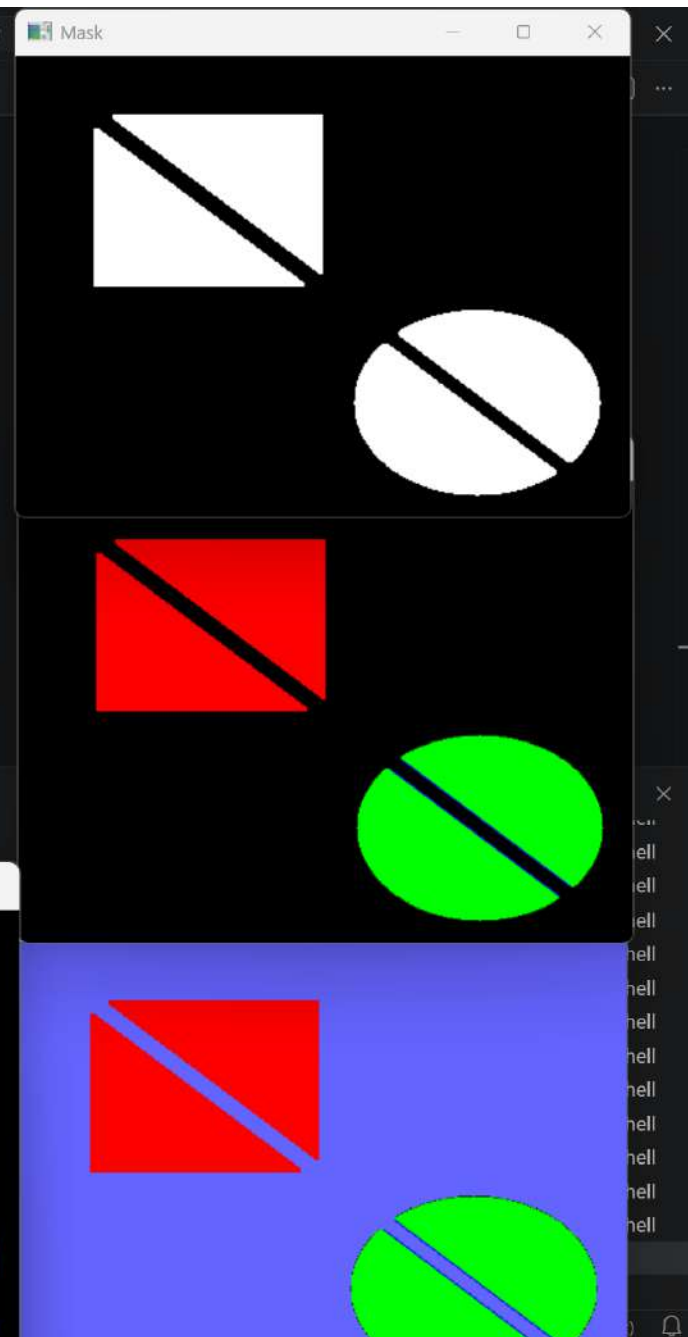
- PYTHON_TEST
 - day8_color_spaces.py
 - day9_histograms.py
 - day10_smoothing_blu...
 - day11_thresholding.py
 - dissolve.mp4
 - drawing_basics.py
 - drawing.py
 - edge_preserving_com...
 - enhanced_image.png
 - equalization_comparis...
 - equalize.py
 - equalized_image.png
 - exercise_1_channel_art...
 - exercise_1_custom_pip...
 - exercise_1_histogram...
 - exercise_1_interactive...
 - exercise_1_threshold...
 - exercise_2_color_picke...
 - exercise_2_compariso...
 - exercise_2_document...
 - exercise_2_equalizatio...
 - exercise_2_noise_redu...
 - exercise_3_d_color_hi...
 - exercise_3_color_isolat...
 - exercise_3_custom_ker...
 - exercise_3_multi_objec...
 - exercise_4_backgroun...
 - exercise_4_color_spac...
 - exercise_4_edge_prese...
 - exercise_4_histogram...
 - exercise_5_contrast_en...
 - exercise_5_deblurring...
 - exercise_1_brightness...

exercise_4_background_removal.py > ...

```
1 import cv2
2 import numpy as np
3
4 def remove_background(image, threshold_method="otsu"):
5     """
6     Remove background from image using thresholding
7     """
8     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
9     blurred = cv2.GaussianBlur(gray, (5, 5), 0)
10
11     if threshold_method == "otsu":
12         mask = cv2.threshold(
13             blurred,
14             0,
15             255,
16             cv2.THRESH_BINARY + cv2.THRESH_OTSU
17         )
18
19     elif threshold_method == "adaptive":
20         mask = cv2.adaptiveThreshold(
21             blurred,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python exercise_4_background_removal.py



File Edit Selection View Go Run Terminal Help python_test

EXPLORER

- PYTHON_TEST
 - exercise1.py
 - exercise1.thumbnail.py
 - exercise2.day3.py
 - exercise2.dissolve.py
 - exercise2.panorama.py
 - exercise2.py
 - exercise2.target.py
 - exercise3.animation.py
 - exercise3.chessboard.py
 - exercise3.day3.py
 - exercise3.py
 - exercise3.toolmask.py
 - exercise4.annotations....
 - exercise4.day3.py
 - exercise4.smart.py
 - exercise4.splash.py
 - exercise5_license_plat...
 - exercise5.alignment.py
 - exercise5.clock.py
 - exercise5.mask.py
 - final_collage.png
 - flipping.py
 - getting_and_setting.py
 - gradient.png
 - grayscale_histogram.p...
 - grayscale_histogram.py
 - histogram_equalizatio...
 - histogram_search_de...
 - histogram_with_mask....
 - house.png
 - hsv_color_wheel.png
 - image.enhancement.c...
 - image.py
- OUTLINE
- TIMELINE

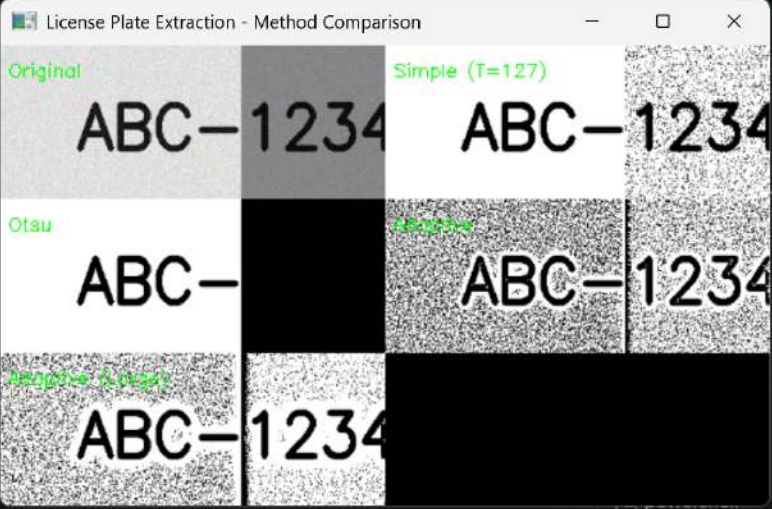
Welcome exercise5_license_plate.py X

```
1 import cv2
2 import numpy as np
3
4 def extract_license_plate(image):
5     """
6     Simulate license plate extraction using thresholding
7     """
8     gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
9
10    methods = {}
11
12    # Method 1: Simple threshold
13    methods["Simple (T=127)"] = cv2.threshold(
14        gray, 127, 255, cv2.THRESH_BINARY
15    )
16
17    # Method 2: Otsu
18    methods["Otsu"] = cv2.threshold(
19        gray, 0, 255,
20        cv2.THRESH_BINARY + cv2.THRESH_OTSU
21    )
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\raghu\OneDrive\Desktop\python_test> python exercise5_license_plate.py

License Plate Extraction - Method Comparison



Simulated License Plate

